arXiv:2310.08560v2 [cs.AI] 12 Feb 2024

MemGPT: Towards LLMs as Operating Systems

Charles Packer [1] **Sarah Wooders** [1] **Kevin Lin** [1]
Vivian Fang [1] **Shishir G. Patil** [1] **Ion Stoica** [1] **Joseph E. Gonzalez** [1]

Abstract

Large language models (LLMs) have revolutionized AI, but are constrained by limited context windows, hindering their utility in tasks like extended conversations and document analysis. To enable using context beyond limited context windows, we propose *virtual context management*, a technique drawing inspiration from hierarchical memory systems in traditional operating systems which provide the illusion of an extended virtual memory via paging between physical memory and disk. Using this technique, we introduce MemGPT (MemoryGPT), a system that intelligently manages different storage tiers in order to effectively provide extended context within the LLM's limited context window. We evaluate our OS-inspired design in two domains where the limited context windows of modern LLMs severely handicaps their performance: document analysis, where MemGPT is able to analyze large documents that far exceed the underlying LLM's context window, and multi-session chat, where MemGPT can create conversational agents that remember, reflect, and evolve dynamically through long-term interactions with their users. We release MemGPT code and data for our experiments at https://research.memgpt.ai.

1. Introduction

In recent years, large language models (LLMs) and their underlying transformer architecture (Vaswani et al., 2017; Devlin et al., 2018; Brown et al., 2020; Ouyang et al., 2022) have become the cornerstone of conversational AI and have led to a wide array of consumer and enterprise applications. Despite these advances, the limited fixed-length context windows used by LLMs significantly hinders their applicability to long conversations or reasoning about long documents. For example, the most widely used open-source LLMs can only support a few dozen back-and-forth messages or reason about a short document before exceeding their maximum input length (Touvron et al., 2023).

Directly extending the context length of transformers incurs a quadratic increase in computational time and memory cost due to the transformer architecture's self-attention mechanism, making the design of new long-context architectures a pressing research challenge (Dai et al., 2019; Kitaev et al., 2020; Beltagy et al., 2020). While developing longer models is an active area of research (Dong et al., 2023), even if we could overcome the computational challenges of context scaling, recent research shows that long-context models struggle to utilize additional context effectively (Liu et al., 2023a). As consequence, given the considerable resources needed to train state-of-the-art LLMs and diminishing returns of context scaling, there is a critical need for alternative techniques to support long context.

In this paper, we study how to provide the illusion of an infinite context while continuing to use fixed-context models. Our approach borrows from the idea of virtual memory paging that was developed to enable applications to work on datasets that far exceed the available memory by paging data between main memory and disk. We leverage the recent progress in function calling abilities of LLM agents (Schick et al., 2023; Liu et al., 2023b) to design MemGPT, an OS-inspired LLM system for **virtual context management**. Using function calls, LLM agents can read and write to external data sources, modify their own context, and choose when to return responses to the user.

These capabilities allow LLMs to effective "page" in and out information between context windows (analogous to "main memory" in operating systems) and external storage, similar to hierarchical memory in traditional OSes. In addition, function calls can be leveraged to manage control flow between context management, response generation, and user interactions. This allows for an agent to choose to *iteratively* modify what is in its context for a single task, thereby more effectively utilizing its limited context.

In MemGPT, we treat context windows as a constrained memory resource, and design a memory hiearchy for LLMs analogous to memory tiers used in traditional OSes (Patterson et al., 1988). Applications in traditional OSes interact

[1]University of California, Berkeley. Correspondence to: Charles Packer <cpacker@berkeley.edu>.

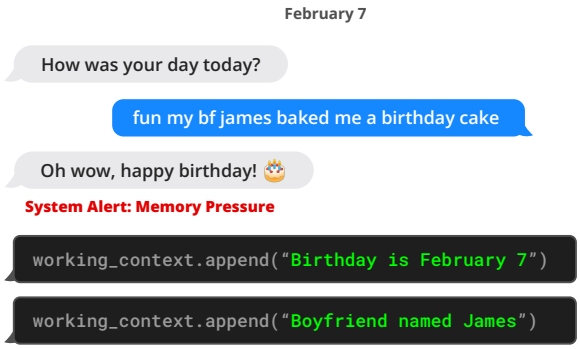

Figure 1. MemGPT (left) writes data to persistent memory after it receives a system alert about limited context space.

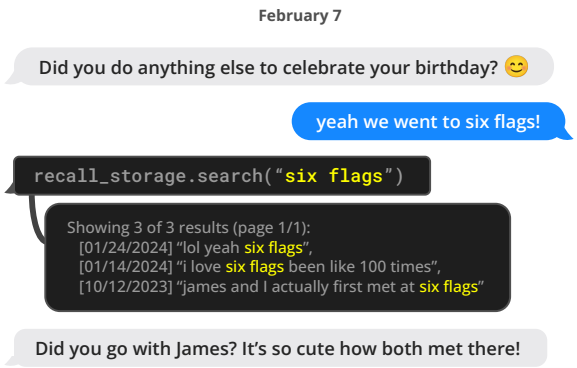

Figure 2. MemGPT (left) can search out-of-context data to bring relevant information into the current context window.

with *virtual memory*, which provides an illusion of there being more memory resources than are actually available in physical (i.e., main) memory by the OS paging overflow data to disk and retrieving data (via a page fault) back into memory when accessed by applications. To provide a similar illusion of longer context length (analogous to virtual memory), we allow the LLM to manage what is placed in its own context (analogous to physical memory) via an 'LLM OS', which we call MemGPT. MemGPT enables the LLM to retrieve relevant historical data missing from what is placed in-context, and also evict less relevant data from context and into external storage systems. Figure 3 illustrates the components of MemGPT.

The combined use of a memory-hierarchy, OS functions and event-based control flow allow MemGPT to handle unbounded context using LLMs that have finite context windows. To demonstrate the utility of our new OS-inspired LLM system, we evaluate MemGPT on two domains where the performance of existing LLMs is severely limited by finite context: document analysis, where the length of standard text files can quickly exceed the input capacity of modern LLMs, and conversational agents, where LLMs bound by limited conversation windows lack context awareness, persona consistency, and long-term memory during extended conversations. In both settings, MemGPT is able to overcome the limitations of finite context to outperform existing LLM-based approaches.

2. MemGPT (MemoryGPT)

MemGPT's OS-inspired multi-level memory architecture delineates between two primary memory types: **main context** (analogous to main memory/physical memory/RAM) and **external context** (analogous to disk memory/disk storage). Main context consists of the LLM *prompt tokens*—anything in main context is considered *in-context* and can be accessed by the LLM processor during inference. External context refers to any information that is held outside of the LLMs fixed context window. This *out-of-context* data must always be explicitly moved into main context in order for it to be passed to the LLM processor during inference. MemGPT provides function calls that the LLM processor to manage its own memory without any user intervention.

2.1. Main context (*prompt tokens*)

The prompt tokens in MemGPT are split into three contiguous sections: the **system instructions**, **working context**, and **FIFO Queue**. The system instructions are read-only (static) and contain information on the MemGPT control flow, the intended usage of the different memory levels, and instructions on how to use the MemGPT functions (e.g. how to retrieve out-of-context data). Working context is a fixed-size read/write block of unstructured text, writeable only via MemGPT function calls. In conversational settings, working context is intended to be used to store key facts, preferences, and other important information about the user and the persona the agent is adopting, allowing the agent to converse fluently with the user. The FIFO queue stores a rolling history of messages, including messages between the agent and user, as well as system messages (e.g. memory warnings) and function call inputs and outputs. The first index in the FIFO queue stores a system message containing a recursive summary of messages that have been evicted from the queue.

2.2. Queue Manager

The queue manager manages messages in **recall storage** and the **FIFO queue**. When a new message is received by the system, the queue manager appends the incoming messages to the FIFO queue, concatenates the prompt tokens and triggers the LLM inference to generate LLM output (the completion tokens). The queue manager writes both the incoming message and the generated LLM output to recall storage (the MemGPT message database). When messages in recall storage are retrieved via a MemGPT function call, the queue manager appends them to the back of

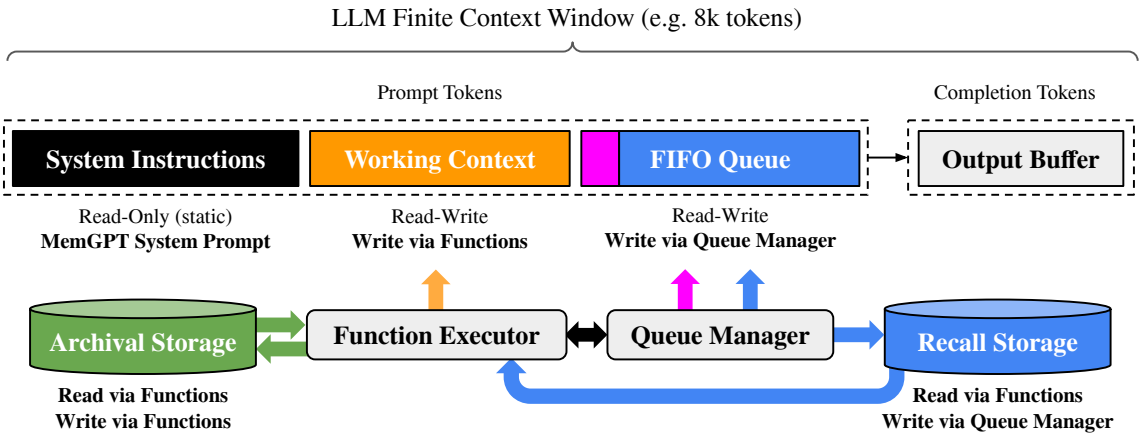

Figure 3. In MemGPT, a fixed-context LLM processor is augmented with a hierarchical memory system and functions that let it manage its own memory. The LLM's prompt tokens (inputs), or *main context*, consist of the system instructions, working context, and a FIFO queue. The LLM completion tokens (outputs) are interpreted as function calls by the function executor. MemGPT uses functions to move data between main context and *external context* (the archival and recall storage databases). The LLM can request immediate follow-up LLM inference to chain function calls together by generating a special keyword argument (`request_heartbeat=true`) in its output; function chaining is what allows MemGPT to perform multi-step retrieval to answer user queries.

the queue to reinsert them into the LLM's context window.

The queue manager is also responsible for controlling context overflow via a queue eviction policy. When the prompt tokens exceed the 'warning token count' of the underlying LLM's context window (e.g. 70% of the context window), the queue manager inserts a system message into the queue warning the LLM of an impending queue eviction (a 'memory pressure' warning) to allow the LLM to use MemGPT functions to store important information contained in the FIFO queue to working context or **archival storage** (a read/write database storing arbitrary length text objects). When the prompt tokens exceed the 'flush token count' (e.g. 100% of the context window), the queue manager flushes the queue to free up space in the context window: the queue manager evicts a specific count of messages (e.g. 50% of the context window), generates a new recursive summary using the existing recursive summary and evicted messages. Once the queue is flushed, the evicted messages are no longer in-context and immediately viewable to the LLM, however they are stored indefinitely in recall storage and readable via MemGPT function calls.

2.3. Function executor (handling of *completion tokens*)

MemGPT orchestrates data movement between main context and external context via function calls that are generated by the LLM processor. Memory edits and retrieval are entirely self-directed: MemGPT autonomously updates and searches through its own memory based on the current context. For instance, it can decide when to move items between contexts (e.g. when the conversation history is becoming too long, as show in Figure 1) and modify its main context to better reflect its evolving understanding of its current objectives and responsibilities (as shown in Figure 3). We implement self-directed editing and retrieval by providing explicit instructions within the system instructions that guide the LLM on how to interact with the MemGPT memory systems. These instructions comprise two main components: (1) a detailed description of the memory hierarchy and their respective utilities, and (2) a function schema (complete with their natural language descriptions) that the system can call to access or modify its memory.

During each inference cycle, LLM processor takes main context (concatenated into a single string) as input, and generates an output string. This output string is parsed by MemGPT to ensure correctness, and if the parser validates the function arguments the function is executed. The results, including any runtime errors that occur (e.g. trying to add to main context when it is already at maximum capacity), are then fed back to the processor by MemGPT. This feedback loop enables the system to learn from its actions and adjust its behavior accordingly. Awareness of context limits is a key aspect in making the self-editing mechanism work effectively, to this end MemGPT prompts the processor with warnings regarding token limitations to guide its memory management decisions. Additionally, our memory retrieval mechanisms are designed to be cognizant of these token constraints and implement pagination to prevent retrieval calls from overflowing the context window.